

Rahnuma

रहनुमा رہنما

A working guide to the ILM, PANINI and Zistgah stack, from bytes and scripts to compilers, Pāṇini and quantum substrates

प्रकृति

	written	Devanagari hub	Romenagri	and back
Devanagari	प्रकृति	→ प्रकृति	→ prak_ri_ti	→ प्रकृति
Telugu	ప్రకృతి	→ प्रकृति	→ prak_ri_ti	→ ప్రకృతి
Bengali	প্রকৃতি	→ प्रकृति	→ prak_ri_ti	→ the same identity
Urdu, as usually written	ہندوی	→ हनदवी	→ _hana_dawee	→ हनदवी
Urdu, with zer and sukun	ہندوی	→ हिन्दवी	→ _hin_dawee	→ हिन्दवी

One identity, many scripts. Telugu and Bengali meet Devanagari in one hub, become the same Romenagri word, and come back exactly. Urdu meets the hub at what is written: without its short-vowel marks the word reads hanadawee; with zer and sukun it is hindawi. Every cell here was produced by the Hindawi tools while this guide was built.

Abhishek Choudhary, AyeAI. Version 2.0.0, 3 October 2026.

Rahnuma, the edition for the field: Engineering, manufacturing and construction

Contents

Part I: Orientation 2

- 1. How to use this guide 3
- 3. Start here: find your way in 4

Part III: Setting up 6

- 15. Running every component 7

Part IV: Foundations 19

- 21. Compilers, from lexer to microcode 20
- 22. Below the software: logic, chips, ternary and qubits 32
- 23. Quantum computing and PANINIq, with the mathematics 35

Appendices 39

- D. For researchers in science, technology, engineering and mathematics 40

Part I: Orientation

1. How to use this guide

In plain words. This guide explains a large body of work on computers, languages and intelligence. You can read it, run the programs on your own computer, or ask an AI to help you, and every example in it was really run.

This guide is for anyone who wants to use, check or extend the work gathered in the *zistgah*, *project-ilm* and *hindawiai* organisations on GitHub: students preparing for GATE, engineers who write compilers, firmware or chip layouts, linguists who work with scripts and sound, and researchers in quantum computing and AI. It asks for curiosity and a laptop, and nothing else.

There are four ways in.

- **Understand.** Part II gives the whole architecture first: where the work comes from, how the parts compose, each term with its nearest analogue and the difference, and the three Acts.
- **Read.** Parts IV and V explain the ideas the work rests on, with the mathematics written out: how numbers and letters sit in memory, what makes a transliteration reversible, how Pāṇini's grammar computes, how a compiler turns text into machine instructions, and how an oscillator network and a quantum circuit attack the same problem.
- **Run.** Part III gives the exact commands to set up Linux, Windows with WSL, or macOS, and to run every component of the estate on your own machine.
- **Ask.** Chapter 24 shows how to put any AI assistant to work on these repositories, and how to check what it tells you.

Conventions used throughout:

- Every example that could run while this guide was being built did run, and the text shows its real output, marked *Executed while this guide was built*. The package's own gate, `bash ops/verify.sh`, runs those examples again and fails if any output differs from what is printed here.
- Status words are used exactly. **Tested:** a test in the repository asserts it. **Executed:** it ran and its output is shown. **Written:** the code exists but has not been run here. **Mocked:** a stand-in replaces hardware that is not attached. **Planned:** described, not built.
- Everything said about the repositories was retrieved from the repositories themselves on 29 September 2026: their listings on GitHub, their `CITATION.cff` files and their READMEs. Where a repository does not record something, this guide says so instead of filling the gap.
- A dagger, †, marks a statement that follows the author's working notes and write-ups as supplied on 29 September and 3 October 2026, rather than the repositories.
- Sanskrit is transliterated in IAST. The spelling is British.

The work lives at github.com/zistgah, github.com/project-ilm and github.com/hindawiai, on the site zistgah.org, and in the Zenodo records listed in Chapter 12.

3. Start here: find your way in

In plain words. Four steps to find your part of the guide: who you are, your board or field, what you want to do, and your path with a PDF made for you.

This guide serves a school student and a senior researcher; a village health worker and an investor. Nobody needs all of it. Four steps find your part, on the site or with this page:

1. **Who you are.** A school student, a college student, a postgraduate or doctoral researcher; a teacher; an academic researcher; a working professional, someone in public service, policy or law, someone in community, NGO or village work, an artist, writer or maker; an industry partner or investor; or an AI agent working for a person.
2. **Your board or your field.** A school student chooses a board, ICSE, ISC, CBSE, IGCSE, IB, a state board or another; and a class. Everyone else chooses a field from the international standard classification of fields of study, ISCED-F 2013, the same one FAKIR's lattice uses. It places each subject where its own scholars would: political science is under social and behavioural sciences, not the humanities; law is under business, administration and law; linguistics is under languages.
3. **What you want to do.** Learn the basics, run the software, study or research, teach, contribute, or adopt, fund or invest.
4. **Your path and your edition.** The site lists your chapters in order, with reading times and the components to try first, and offers a PDF edition made for you. Every edition below is a printed selection from this one guide, so all of them stay in step.

Edition	PDF
School student	role-school.pdf
College student	role-college.pdf
Postgraduate or doctoral researcher	role-pg.pdf
Teacher or lecturer	role-teacher.pdf
Academic researcher	role-researcher.pdf
Working professional or engineer	role-professional.pdf
Public service, policy or law	role-public.pdf
Community, NGO or village work	role-community.pdf
Artist, writer or maker	role-creator.pdf
Industry partner, investor or funder	role-industry.pdf
Generic programmes and qualifications	field-00.pdf
Education	field-01.pdf
Arts and humanities	field-02.pdf
Social sciences, journalism and information	field-03.pdf

Edition	PDF
Business, administration and law	field-04.pdf
Natural sciences, mathematics and statistics	field-05.pdf
Information and Communication Technologies (ICTs)	field-06.pdf
Engineering, manufacturing and construction	field-07.pdf
Agriculture, forestry, fisheries and veterinary	field-08.pdf
Health and welfare	field-09.pdf
Services	field-10.pdf

The first public version of this guide, written for advanced researchers and senior professionals, stays as it was at [version 1](#).

Part III: Setting up

15. Running every component

In plain words. Step-by-step commands to start each part on your own computer, with what you should see when it works and what to do when it does not.

Every component below was cloned and run on a clean machine on 3 October 2026, with the commands its own README gives, and each card says exactly what happened. Where a step needed something the README does not mention, the card adds it; where a component did not pass, the card says why. Set up the machine first (Chapters 13 and 14).

Each card has the same shape: what the component is, what it needs, the commands to type in order, what you should see, and its status. Every card starts with a clone of the repository into a folder of the same name; the commands that follow are typed inside that folder.

Machine: Ubuntu 24.04, Python 3.12, Node 22, GCC 13. *Run on:* 3 October 2026.

Language and identity

Hindawi and Romenagri

What it is. The Hindawi Programming System's C front end, its driver hincc and the Romenagri script layer. *Needs:* gcc, make, flex, libfl-dev, gawk.

```
git clone https://github.com/hindawiai/chintamani.git && cd chintamani
mkdir -p .root/usr/bin .root/usr/lib .root/usr/include
make -C Romenagri all && make -C Romenagri install INSTRROOT="$PWD/.root"
export PATH="$PWD/.root/usr/bin:$PATH"
make -C Hindawi/guru all && make -C Hindawi/guru install INSTRROOT="$PWD/.root"
make -C Hindawi/hindr install INSTRROOT="$PWD/.root"
cp Hindawi/samples/HindiC.uhin . && hincc HindiC.uhin
printf 'रम\nरम\n' | ./hin.exe
```

You should see: संकलन के परिणाम, then the program asks आपका नाम क्या है? and counts to ten. *Status:* Executed for this guide. The compilers chapter prints the run.

The Urdu edition

What it is. Urdu shailis and the driver urducc, which sends an Urdu-script program through the Devanagari hub and Romenagri. *Needs:* the Hindawi build above, on the PATH.

```
git clone https://github.com/zistgah/urdu-ilm.git && cd urdu-ilm
mkdir -p .root/usr/bin .root/usr/lib .root/usr/include
make -C Romenagri all && make -C Romenagri install INSTR00T="$PWD/.root"
export PATH="$PWD/.root/usr/bin:$PATH:<chintamani clone>/.root/usr/bin"
make -C ILM/guru all
bash ILM/urducc -s ILM/samples/UrduC.uhin
bash ILM/urducc ILM/samples/UrduC.uhin && ./hin.exe
```

You should see: C with the names na and ka, then the prompt in the hub script. *Status:* Executed for this guide. Replace with the folder of the Hindawi clone; the compilers chapter prints the run.

tajziya

What it is. The parser federation for the classical languages, by family. *Needs:* python3, make.

```
git clone https://github.com/zistgah/tajziya.git && cd tajziya
PYTHONPATH=src python3 -m tajziya commonalities
make check
```

You should see: make check: 19 passed, 0 failed, 1 unjudged. *Status:* Passed when run on 3 October 2026. Run the module with PYTHONPATH=src, from the clone's root.

The PANINI family

PANINI, the prompt-cycle language

What it is. The language the cyclers are written in, with its engine and studio. *Needs:* python3, node.

```
git clone https://github.com/zistgah/panini.git && cd panini
python3 panini.py check cyclers/pench.pni
python3 panini.py stages cyclers/yadein.pni
node tests/panini.test.mjs
python3 panini.py serve
```

You should see: 0 problems; 45 pass, 0 fail; the studio answers on 127.0.0.1:8717. *Status:* Passed when run on 3 October 2026.

PANINI, parser and interpreter

What it is. A general-purpose PANINI implementation with its conformance report. *Needs:* node.

```
git clone https://github.com/zistgah/panini_by_claude.git && cd panini_by_claude
node bin/panini.mjs conformance
npm test
```

You should see: 145 passed, 0 failed. *Status:* Passed when run on 3 October 2026.

PANINI, the stage-0 bootstrap

What it is. PANINI with a bootstrap that records its own evidence. *Needs:* node.

```
git clone https://github.com/zistgah/panini_by_grok.git && cd panini_by_grok
node src/cli.js run examples/hello.pni
node tests/run.mjs
node scripts/bootstrap.mjs
```

You should see: Hello, PANINI; 32 passed, 0 failed; stage-0 evidence written. *Status:* Passed when run on 3 October 2026. Its wider report, npm test, passes 81 of 83 here; the two others need tools this machine lacks.

The Humanesque merged release

What it is. PANINI, ILM, Hindawi, the cyclers and Mez in one tree. *Needs:* bash, node; optionally zig.

```
git clone https://github.com/zistgah/humanesque.git && cd humanesque
bash VERIFY.sh
npm test
```

You should see: 79 of 83 checks pass. *Status:* Partly passes, as run on 3 October 2026. The four that do not pass here need two downloadable model files (wllama.wasm and stories15M.Q4_0.gguf), the zig toolchain, and the Bengali flattening round trip.

PANINIq

What it is. The oscillator substrate, the PEDLER engine and quantum validation. *Needs:* python3, numpy, scipy, qiskit.

```
git clone https://github.com/zistgah/paniniq.git && cd paniniq
python3 -m venv ../venv-paniniq && . ../venv-paniniq/bin/activate
export PYTHONDONTWRITEBYTECODE=1
pip install -r requirements.txt qiskit
python3 -m unittest discover -s tests
python3 demo.py
bash ops/verify.sh
```

You should see: Ran 23 tests, OK; its gate 10 passed. *Status:* Executed for this guide. The first-run chapter prints the run.

PANINIB, biological realization

What it is. Biological intent lowered through typed IRs to a realization plan. *Needs:* python3.

```
git clone https://github.com/zistgah/paninib.git && cd paninib
python3 panini compile examples/replace.panini -o out
python3 panini inspect out --stage sequence-ir
python3 panini verify out
```

You should see: each step prints its JSON record. *Status:* Passed when run on 3 October 2026.

PANINIphy, physical realization

What it is. Physical intent compiled through a physical IR, with parts chosen by requirement. *Needs:* python3.

```
git clone https://github.com/zistgah/paniniphy.git && cd paniniphy
python3 panini compile examples/modular_block.panini -o out
python3 panini parts --category actuator --force 50N --stroke 20mm --voltage 12V
```

You should see: physical resolution=RESOLVED verify=True. *Status:* Passed when run on 3 October 2026.

Cyclers, desk and studio

matba, the press

What it is. Posters in, book out, sealed, pushed, minted. *Needs:* python3.

```
git clone https://github.com/zistgah/matba.git && cd matba
python3 matba.py serve
```

You should see: the press answers on 127.0.0.1:8710. *Status:* Passed when run on 3 October 2026.

khwab, the cutting room

What it is. The visual cycler. *Needs:* python3.

```
git clone https://github.com/zistgah/khwab.git && cd khwab
python3 khwab.py serve
```

You should see: the cutting room answers on 127.0.0.1:8711. *Status:* Passed when run on 3 October 2026.

awaz, the listening room

What it is. The audio cycler. *Needs:* python3.

```
git clone https://github.com/zistgah/awaz.git && cd awaz
python3 awaz.py serve
```

You should see: the listening room answers on 127.0.0.1:8712. *Status:* Passed when run on 3 October 2026.

tilasm, pench and yadein

What it is. The immersive, embodied and record cyclers; each repository has the same tests and its own page. *Needs:* node.

```
git clone https://github.com/zistgah/tilasm.git && cd tilasm
node tests/cycler.test.mjs
node tests/inbox.test.mjs
node tests/models.test.mjs
```

You should see: 20, 23 and 22 pass, 0 fail. *Status:* Partly passes, as run on 3 October 2026. Run the same three in zistgah/pench and zistgah/yadein. Two further test files, genie.test.mjs and leak.test.mjs, import modules the repositories do not carry.

GENIE

What it is. The Generalized Emotive-Narrative Interaction Engine. *Needs:* node.

```
git clone https://github.com/zistgah/genie.git && cd genie
node test/run.js
```

You should see: 191 passed, 0 failed. *Status:* Passed when run on 3 October 2026. Open index.html, or its page, for the engine itself.

alam

What it is. Small work, cycled with any AI. *Needs:* node, npm.

```
git clone https://github.com/zistgah/alam.git && cd alam
npm install jsdom
node test/alam-test.js
node test/dom-test.js
```

You should see: alam dom: 9 passed, 0 failed. *Status:* Passed when run on 3 October 2026. The page test needs jsdom, installed by the first line.

Mez, the Cognitive Workbench

What it is. The local-first desk. *Needs:* python3.

```
git clone https://github.com/zistgah/mez.git && cd mez
./mez doctor
./mez bearings
./mez serve
```

You should see: no account, no API key, no network required; the desk answers on 127.0.0.1:7373. *Status:* Passed when run on 3 October 2026.

Knowledge, domains and instruments

FAKIR

What it is. The knowledge kernel and its explorer. *Needs:* bash.

```
git clone https://github.com/zistgah/fakir.git && cd fakir
bash ops/verify.sh
```

You should see: CONTRACT SATISFIED, 6 checks. *Status:* Passed when run on 3 October 2026. The explorer is its page: zistgah.org/fakir

Dhancha

What it is. The domain spine every FAKIR domain is built on. *Needs:* bash, python3, make.

```
git clone https://github.com/zistgah/dhancha.git && cd dhancha
make check
```

You should see: site matches the registries; pr_gate bites on a claim and passes a clean change. *Status:* Passed when run on 3 October 2026.

ertabat

What it is. The communication-engineering domain. *Needs:* bash.

```
git clone https://github.com/zistgah/ertabat.git && cd ertabat
bash ops/verify.sh
```

You should see: CONTRACT OK. *Status:* Passed when run on 3 October 2026.

jugaad28

What it is. The 28 nm tape-out catalogue. *Needs:* bash.

```
git clone https://github.com/zistgah/jugaad28.git && cd jugaad28
bash ops/verify.sh
```

You should see: VERIFY GREEN. *Status:* Passed when run on 3 October 2026.

TransEg

What it is. The local digital-twin reference implementation. *Needs:* bash, python3; Docker for the stand-up.

```
git clone https://github.com/zistgah/transeg.git && cd transeg
./scripts/test.sh
./scripts/bootstrap.sh --dry-run
```

You should see: 10 passed; the stand-up plan. *Status:* Passed when run on 3 October 2026. The full stand-up, ./scripts/bootstrap.sh and docker compose, needs Docker and a GPU; it was not run here.

TransEg identity governance

What it is. Typed layers, envelope, staged policies and audit. *Needs:* python3, jsonschema, pytest.

```
git clone https://github.com/zistgah/transeg-idgov.git && cd transeg-idgov
pip install jsonschema pytest
python3 tools/idgov_validate.py policies
python3 -m pytest -q tests/
```

You should see: VALID: 8 stage policies; 14 passed. *Status:* Passed when run on 3 October 2026.

CHAKRA

What it is. The offline observatory and its calendars. *Needs:* node.

```
git clone https://github.com/project-ilm/chakra.git && cd chakra
node test/run.js
```

You should see: ALL SUITES PASSED. *Status:* Passed when run on 3 October 2026. Open index.html for the observatory.

Jyotish

What it is. The panchang, the Kaivalyik projection of CHAKRA. *Needs:* bash, node.

```
git clone https://github.com/zistgah/jyotish.git && cd jyotish
bash ops/verify.sh
```

You should see: PASSED, with 1 check skipped and named. *Status:* Passed when run on 3 October 2026. Open index.html for the panchang.

Cognition and hardware

PRATIK kernel

What it is. The balanced-ternary, event-driven kernel. *Needs:* g++; nvcc for CUDA.

```
git clone https://github.com/zistgah/pratik_core_mvp.git && cd pratik_core_mvp
bash build.sh
bash build.sh cuda
```

You should see: ALL CHECKS PASSED. *Status:* Passed when run on 3 October 2026. The CPU build passed here; the CUDA build needs nvcc and an NVIDIA GPU. The script is not marked executable, so call it with bash.

Zamin

What it is. Ternary silicon: SPICE cells, layout and a simulator. *Needs:* ngspice, python3.

```
git clone https://github.com/zistgah/zamin.git && cd zamin
cd spice && ngspice -b sign_frustration_tb.sp
```

You should see: a SPICE run of the ternary cells. *Status:* Does not run as is, as run on 3 October 2026. On ngspice 42 the testbench stops: the source v_gnd is reported as a shorted voltage source. The interactive simulator is its page.

QEDLER

What it is. The event-hypergraph physics programme. *Needs:* bash.

```
git clone https://github.com/project-ilm/qedler.git && cd qedler
bash ops/verify.sh
bash ops/resume.sh && cat RESUME.md
```

You should see: 5 of 6 clauses pass. *Status:* Partly passes, as run on 3 October 2026. Clause C2, manifest completeness, fails: 15 files are not in MANIFEST.sha256. Resealing the manifest clears it.

Provenance, tools and labs

Misty DOI

What it is. DOI minting and packaging for Zenodo. *Needs:* python3, pip.

```
git clone https://github.com/project-ilm/misty-doi.git && cd misty-doi
pip install misty-doi
misty --help
misty init
misty validate -m misty.json
```

You should see: `usage: misty [-h] [--version] {init,validate,transform,package,ots,publish}`. *Status:* Passed when run on 3 October 2026. Publishing needs a Zenodo token in ZENODO_TOKEN.

Research Kundali

What it is. A research map of a person, a lab or a paper. *Needs:* python3 and network access to the public records it reads.

```
git clone https://github.com/project-ilm/research-kundali.git && cd research-kundali
python3 kundali/kundali.py <ORCID, lab name or DOI> --out out
```

You should see: a research map in out/. *Status:* Not run here: needs network access. It reads public records such as ORCID; it could not reach them from this build machine, and stops rather than write an empty chart.

The linguistics lab

What it is. A starter lab in a container. *Needs:* Docker.

```
git clone https://github.com/project-ilm/linguistics-labs.git && cd linguistics-labs
docker build -t ilm-lab .
docker run --rm -p 8888:8888 -v "$PWD":/lab ilm-lab
```

You should see: the lab on localhost:8888. *Status:* Not run here: needs Docker. Not run here: this build machine has no Docker.

This guide

What it is. The site, the PDF and the Markdown, with their gate. *Needs:* python3, gcc, make, flex, libfl-dev, gawk, gdb; for a rebuild also pandoc, node, graphviz.

```
git clone https://github.com/zistgah/rahnuma.git && cd rahnuma
bash ops/verify.sh
bash examples/hindawi.sh words
python3 tools/build.py
```

You should see: 12 passed, 0 failed, 0 unjudged. *Status:* Passed when run on 3 October 2026.

Components you open in a browser

Component	Page
FAKIR's explorer	zistgah.org/fakir
The shared dome	zistgah.github.io/dome
AAB, the factory process	zistgah.org/aab
GENIE	zistgah.github.io/genie

tilasm, pench and yadein | zistgah.github.io/tilasm |
Kitab | zistgah.github.io/kitab |
Tok DOI | project-ilm.github.io/tok-doi |
CHAKRA | project-ilm.github.io/chakra |
Jyotish | zistgah.github.io/jyotish |
ILM | ilm.codes |
PANINI's studio | zistgah.github.io/panini |
The estate | zistgah.org |

Part IV: Foundations

21. Compilers, from lexer to microcode

In plain words. How a program written in Hindi or Urdu becomes instructions a machine can run, with every name kept intact all the way to the debugger.

A compiler is a chain of translations, each into a representation closer to the machine: source text, tokens, a syntax tree, a checked tree, an intermediate representation, optimised intermediate code, assembly, an object file, an executable, a running process, and finally, inside the processor, micro-operations.

Lexing: regular languages and automata

Tokens are described by regular expressions. Thompson's construction turns a regular expression of length n into a nondeterministic finite automaton with at most $2n$ states; the subset construction turns that into a deterministic automaton whose states are sets of the original states, up to 2^n in the worst case and few in practice; Hopcroft's algorithm minimises the result in $O(n \log n)$ time.

lex (Lesk and Schmidt, Bell Labs, 1975) generates such a scanner in C from a list of patterns and actions; flex is the fast, free lex that Linux systems carry. flex works on bytes, so a keyword written in Devanagari is simply a pattern of UTF-8 bytes, and the whole Devanagari block is the pattern

`\xE0\xA4[\x80-\xBF]|\xE0\xA5[\x80-\xBF]`, whose end points the output in Chapter 18 printed.

Parsing: context-free grammars

A grammar $G = (N, \Sigma, P, S)$ has nonterminals, terminals, productions and a start symbol. The classic expression grammar is

$$\begin{aligned} E &\rightarrow T E' & E' &\rightarrow + T E' \mid \varepsilon \\ T &\rightarrow F T' & T' &\rightarrow * F T' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

A predictive, LL(1), parser chooses a production by looking at one token, guided by two families of sets. With $\neg$ marking the end of input:

$$\begin{aligned} \text{FIRST}(E) &= \text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \} \\ \text{FIRST}(E') &= \{ +, \varepsilon \}, \quad \text{FIRST}(T') = \{ *, \varepsilon \} \\ \text{FOLLOW}(E) &= \text{FOLLOW}(E') = \{), \neg \} \\ \text{FOLLOW}(T) &= \text{FOLLOW}(T') = \{ +,), \neg \} \\ \text{FOLLOW}(F) &= \{ +, *,), \neg \} \end{aligned}$$

LR parsing (Knuth, 1965) reads left to right, shifting tokens onto a stack and reducing them when a production's right side is on top, which builds a rightmost derivation in reverse. LALR(1) (DeRemer, 1969) merges LR(1) states that share a core, which keeps the tables small enough for practice. yacc (Johnson, Bell Labs) generates LALR(1) parsers in C, and GNU Bison is its compatible successor; the name is a joke on yacc, since a yak and a bison are cousins. When a grammar allows two moves, the generator reports a shift-reduce or reduce-reduce conflict, and declarations such as `%left '+' '-'`

and `%left '*' '/'` settle it by precedence and associativity: a rule-ordering device of the kind Pāṇini's 1.4.2 states for his grammar. The standard example is the dangling else, `if a then if b then s1 else s2`, which has two parse trees; yacc resolves it by shifting, so the `else` joins the nearest `if`.

Meaning: symbol tables and syntax-directed translation

After parsing, names are resolved through symbol tables, one per scope, and types are checked. An attribute grammar attaches values to the nodes of the tree: synthesised attributes flow up from children, inherited attributes flow down from parents and siblings, and definitions that use only synthesised attributes, or inherited attributes from the left, can be evaluated during parsing itself. This is what the GATE syllabus calls syntax-directed translation.

Intermediate representation

Three-address code has at most one operator per instruction:

```
t1 = x * 4
t2 = t1 + 6
return t2
```

Instructions without internal jumps form basic blocks, and the blocks form a control-flow graph. In static single assignment form every variable is assigned exactly once; where control paths join, a φ -function chooses the value from whichever predecessor was taken, $x_3 = \varphi(x_1, x_2)$. SSA is built by placing φ -functions at the dominance frontiers of the definitions (Cytron and colleagues, 1991). GCC's middle end works on GIMPLE in SSA form; LLVM's IR is SSA throughout.

Dataflow analysis as a fixed point

A variable is live at a point if its current value may still be read. For each instruction n ,

$$in[n] = use[n] \cup (out[n] \setminus def[n]), \quad out[n] = \bigcup_{s \in succ(n)} in[s].$$

Start with every set empty and apply the equations until nothing changes. The sets form a lattice ordered by inclusion, the equations are monotone, and the lattice has finite height, so the iteration stops at the least fixed point. Reaching definitions, available expressions and constant propagation are the same pattern over different lattices.

Optimisation, watched on a real compiler

The usual transformations are constant folding and propagation, common-subexpression elimination, dead-code elimination, moving loop-invariant code out of loops, strength reduction (a multiplication by four becomes a shift or an address computation), inlining, unrolling and vectorisation.

File: `examples/fold.c`

```

/* examples/fold.c: two small functions, to watch an optimising back end at work.
   © 1993-2026 Abhishek Choudhary. All rights reserved. AyeAI.
   SPDX-License-Identifier: GPL-3.0-or-later */
int scale(int x) { int k = 2 * 3; return x * 4 + k; }
int sum_to(int n) { int s = 0; for (int i = 1; i <= n; i++) s += i; return s; }

```

Captured while this guide was built: `gcc -O2 -S -masm=intel -fno-asynchronous-unwind-tables -fcf-protection=none -fno-ident -o - examples/fold.c | sed -n '/^scale:/,/size.sum_to/p'`

```

scale:
    lea eax, 6[0+rdi*4]
    ret
    .size    scale, .-scale
    .p2align 4
    .globl   sum_to
    .type    sum_to, @function
sum_to:
    test    edi, edi
    jle     .L6
    lea     ecx, 1[rdi]
    xor     edx, edx
    and     edi, 1
    mov     eax, 1
    je      .L5
    mov     eax, 2
    mov     edx, 1
    cmp     eax, ecx
    je      .L3
    .p2align 4,,10
    .p2align 3
.L5:
    lea     edx, 1[rdx+rax*2]
    add     eax, 2
    cmp     eax, ecx
    jne     .L5
.L3:
    mov     eax, edx
    ret
    .p2align 4,,10
    .p2align 3
.L6:
    xor     edx, edx
    mov     eax, edx
    ret
    .size    sum_to, .-sum_to

```

`scale` became a single instruction. `lea eax, 6[0+rdi*4]` computes $4x + 6$ in the address unit: the constant 2×3 was folded, and the multiplication by four was reduced to scaled addressing. In `sum_to`, GCC peeled one iteration when n is odd and then unrolled the loop by two, adding i and $i + 1$ in one step as $2i + 1$ with another `lea`. The exact output depends on the compiler's version and target; this one was produced by the GCC recorded in the package's `docs/BUILD.json`.

Registers, instructions and scheduling

Register allocation is graph colouring. Each variable's live range is a node; two nodes are joined when the variables are live at the same time; colouring the graph with k colours assigns k registers. Deciding whether a graph is k -colourable is NP-complete for $k \geq 3$, so compilers use heuristics: Chaitin's simplify-and-spill (1982), Briggs's optimistic colouring, and linear scan in just-in-time compilers where speed of compilation matters most. Instruction selection tiles the IR tree with machine instructions, instruction scheduling reorders them to hide latencies, and a peephole pass cleans up short sequences.

Assembling, linking, loading

The assembler writes an object file, ELF on Linux, with sections (`.text` , `.data` , `.rodata` , `.bss`), a symbol table, and relocations marking the places whose final addresses are not yet known. The linker resolves symbols across objects and libraries and applies the relocations; the dynamic loader maps shared libraries into the process when it starts.

Below the instruction set: micro-operations and microcode

On current high-performance processors, architectural instructions are decoded into internal micro-operations, and complex instructions are sequenced from microcode, the lowest layer at which code is still written by people; microcode can be updated after the chip is made. GATE's computer organisation syllabus asks for the design of hardwired and microprogrammed control units, and a microprogrammed control unit is exactly a small engine that runs microcode to drive the datapath.

Front ends and back ends: why a shared back end is not a dependency

With m source languages and n target machines, separate compilers would number mn ; with a shared intermediate representation, m front ends and n back ends suffice, $m + n$ in all. GCC's front ends for C, C++, Objective-C, Fortran, Ada, Go, D, Modula-2 and others all lower to GENERIC and GIMPLE and then RTL, and its back ends serve dozens of processors. LLVM's IR is shared by Clang, Rust, Swift, Julia and others. Nobody says that Rust depends on C++ because LLVM is written in C++: a new front end reusing a mature back end is the ordinary path.

The back end never sees keywords. By the time a program is GIMPLE or RTL, a loop written with `for` and the same loop written with `क्रम` have become the same structure: basic blocks, a condition and a branch. The human language of the keywords is settled and discarded at the front end. That is the precise answer to the objection that a system which uses GCC as its back end depends on English; the same holds for PANINIq's use of Qiskit, where what crosses the boundary is a Hamiltonian, a matrix of complex numbers. What the back end does see is the program's names, which it must carry into the object file intact; that is the script layer's job, and the last sections of this chapter run it.

Bootstrapping and the self-hosting fixed point

A compiler has three languages: the source S it reads, the target T it writes, and the language I it is written in, drawn as a T-shaped tombstone diagram. Diagrams chain: a compiler for X written in C, built by an existing C compiler; runs on the machine; with it you compile a compiler for X written in

X, and from then on the language compiles itself. The test of self-hosting is a fixed point: stage 1, built by the old compiler, builds stage 2; stage 2 builds stage 3; stages 2 and 3 must be identical. GCC’s own `make bootstrap` performs this three-stage build and compares the last two stages.

A reversible transliteration (Chapter 19) plays the part of the first compiler in such a chain. It lets the whole existing toolchain process programs written in the script, and its inverse restores the script in the output and the diagnostics, so the native-script tools can then be brought up the same way. Hindawi’s own keyword lexer was brought up exactly so, through Romenagri, as the next sections show.

Trust: the precise meaning of a sovereign stack

Ken Thompson showed in 1984 that a compiler can be altered to insert a back door into the login program, and to insert the same alteration into every compiler it later compiles, so that the back door survives after both source files have been cleaned. Reading the source is not enough. David Wheeler’s diverse double-compiling (2005, and his dissertation of 2009) answers it: compile the compiler’s source with a second, independently developed compiler; use the result to compile the source again, and compare. If the output is bit-for-bit identical to the original self-compiled binary, that binary corresponds to its source, under stated assumptions.

Together with reproducible builds, in which the same source always yields the same bits, this gives the phrase “sovereign stack” an exact technical meaning: a stack that its users can rebuild from source and verify with independent tools, with people able to do both. Which alphabet the keywords use has nothing to do with it.

Hindawi’s pipeline, run

The Hindawi Programming System has nine *shailis*, each a front end over one host language:

Shaili	Host
गुरु guru	C
श्रेणी shraeni	C++
यंत्र yantra	assembly
कृत्रिम kritrima	Java
प्राथमिक praatha	BASIC
सूची soochee	Python
व्याकरण wyaaka	yacc
रोबोट robot	LOGO
शब्द shabda	lex

A program begins with a line naming its shaili, `<शैली गुरु>` for C, and one driver, `hincc`, compiles every shaili:

```
cat $1 | fixuninum | iconv -f utf-8 -t utf-16 | uni2acii | acii2cf | hincc.awk
```

`fixuninum` turns Devanagari numerals into ASCII digits; `iconv` and `uni2acii` turn Unicode into ACII; `acii2cf` writes the whole program in Romenagri's compiler form, every keyword and every name, leaving string literals as they are; and `hincc.awk` reads the shaili line, which is now `<shailee guru>`, and hands the rest of the program to that shaili's driver. For C that is `gurucc`:

```
cat $1 | acii2uni | iconv -f UTF-16 -t UTF-8 | h2c > tempfil0123.tmphin.c
gcc tempfil0123.tmphin.c -o hin.exe -lm
```

Its `acii2uni` restores the string literals to Unicode, so that the program prints in the script, and `h2c` maps each Romenagri keyword to its C keyword: `poor_nnaa_mka` (पूर्णक) to `int`, `krama` (क्रम) to `for`, `ma_likhoa` (म_लिखो) to `printf`, `mukhya` (मुख्य) to `main`. Every other word is a name, and it reaches GCC in Romenagri. The stages run in the order of the three axes: the script first, in `hincc`; then the language, in `h2c`, and for lex and yacc sources in `h2l` and its companions before it; then the standard, in flex and GCC.

The keyword lexer is itself written in Hindi. Its source, `h2c.uhin`, passes through the same `uni2acii` | `acii2cf` before flex compiles it, so the tool that reads Hindi programs is brought up through its own script layer, the bootstrapping step described above. The transducers also compose: a lex or yacc action block is C, so `h2l` carries only what lex adds beyond C, mapping `श_शब्द` to `yylex`, `श_ब_मान` to `yylval`, `श_माला` to `yytext` and `श_पंक्ति` to `yylineno`, and hands the rest to `h2c`. Counted that way, `shabda` has 6 rules of its own and 321 through `h2c`, 327 in all, and `wyaaka`, for yacc, has 341. Reading a composing shaili on its own would report a working system as broken.

The run below builds Romenagri, the guru shaili and `hincc` from the retrieved sources in `vendor/chintamani` with their own Makefiles, then compiles and runs Hindawi's own C sample:

Executed while this guide was built: `bash examples/hindawi.sh sample`

```
$ hincc HindiC.uhin
संकलन के परिणाम
=====

The C that gurucc handed to gcc:
#include <stdio.h>

int main()
{
    char _a[80];
    int ka;
    printf("आपका नाम क्या है?\n");
    scanf("%s", _a);
    printf("नमस्ते %s.\n", _a);
    for(ka=1; ka<=10; ka++)
        printf("%d\n", ka);
    scanf("%s", _a);
    return 0;
}

$ ./hin.exe, with राम typed at both prompts
आपका नाम क्या है?
नमस्ते राम.
```

```
1
2
3
4
5
6
7
8
9
10
```

अ has become `_a` and क has become `ka` ; the strings still print in Devanagari.

The symbol bridge, from source to debugger

This program, in the guru shaili, has a global, a function, a parameter and a local, all named in Hindi:

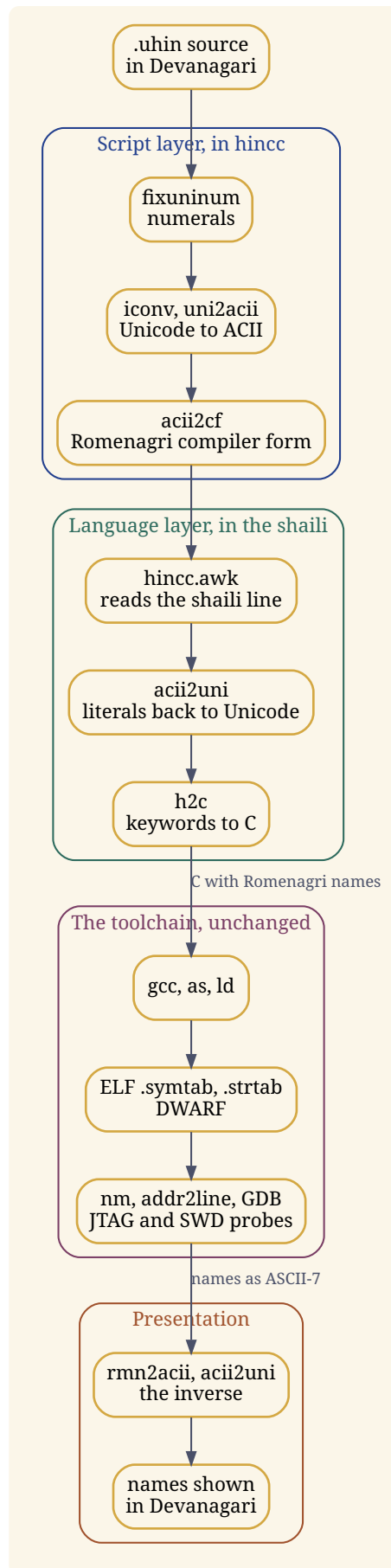


Figure 11. The symbol bridge: a Hindi program through `hincc`, GCC and the debugger; its names carried in Romenagri and rendered back.

File: `examples/yog.uhin`

```
<शैली गुरु>
#समावेश <मानकपन.स>

पूर्णक योग;

पूर्णक जोड़ो(पूर्णक सीमा)
{
    पूर्णांक गिनती;
    क्रम(गिनती=1; गिनती<=सीमा; गिनती++)
        योग = योग + गिनती;
    वापस योग;
}

पूर्णक मुख्य()
{
    म_लिखो("१ से १० तक का योग = %d\n", जोड़ो(10));
    वापस 0;
}
```

Executed while this guide was built: `bash examples/hindawi.sh yog`

```
$ hinc yog.uhin
संकलन के परिणाम
=====

The C that gurucc handed to gcc:
#include <stdio.h>

int yoaga;

int joa_rdoa(int seemaa)
{
    int gina_tee;
    for(gina_tee=1; gina_tee<=seemaa; gina_tee++)
        yoaga = yoaga + gina_tee;
    return yoaga;
}

int main()
{
    printf("1 से 10 तक का योग = %d\n", joa_rdoa(10));
    return 0;
}

$ ./hin.exe
1 से 10 तक का योग = 55
```

योग is `yoaga`, जोड़ो is `joa_rdoa`, सीमा is `seemaa` and गिनती is `gina_tee`: names over A to Z, a to z and the underscore, which GCC, the assembler, the linker and every later tool accept as they are. Below, the same program is built with debugging information and examined with the ordinary tools, and then the same transcript is rendered through the Romenagri inverse. The binary is not touched; only what a person reads changes. The C names that Hindawi has words for, `main` and `printf`, come back through `c2h` as they do in Hindawi's own `std2hin`; the program's own names come back through `rmn2acii` and `acii2uni`.

Captured while this guide was built: `bash examples/hindawi.sh debug`

```
$ nm --defined-only yog-g, the program's own symbols
0000000000001149 T joa_rdoa
0000000000001182 T main
0000000000004014 B yoaga

$ readelf --debug-dump=info yog-g, the names DWARF records
variable yoaga
subprogram printf
subprogram main
subprogram joa_rdoa
formal_parameter seemaa
variable gina_tee

$ gdb -batch: break joa_rdoa, run, backtrace, info args, finish, print yoaga
Breakpoint 1 at 0x1154: file ./tempfil0123.tmphin.c, line 8.

Breakpoint 1, joa_rdoa (seemaa=10) at ./tempfil0123.tmphin.c:8
8      for(gina_tee=1; gina_tee<=seemaa; gina_tee++)
#0  joa_rdoa (seemaa=10) at ./tempfil0123.tmphin.c:8
#1  0x000055555555194 in main () at ./tempfil0123.tmphin.c:15
seemaa = 10
0x000055555555194 in main () at ./tempfil0123.tmphin.c:15
15      printf("1 से 10 तक का योग = %d\n", joa_rdoa(10));
Value returned is $1 = 55
$2 = 55

$ addr2line -f -s -e yog-g 0x0000000000001149
joa_rdoa
tempfil0123.tmphin.c:6

The same, rendered through the Romenagri inverse; the binary is unchanged:
$ nm --defined-only yog-g, the program's own symbols
0000000000001149 T जोड़ो
0000000000001182 T मुख्य
0000000000004014 B योग

$ readelf --debug-dump=info yog-g, the names DWARF records
variable योग
subprogram म_लिखो
subprogram मुख्य
subprogram जोड़ो
formal_parameter सीमा
variable गिनती

$ gdb -batch: break जोड़ो, run, backtrace, info args, finish, print योग
Breakpoint 1 at 0x1154: file ./tempfil0123.tmphin.c, line 8.

Breakpoint 1, जोड़ो (सीमा=10) at ./tempfil0123.tmphin.c:8
8      for(गिनती=1; गिनती<=सीमा; गिनती++)
#0  जोड़ो (सीमा=10) at ./tempfil0123.tmphin.c:8
#1  0x000055555555194 in मुख्य () at ./tempfil0123.tmphin.c:15
सीमा = 10
0x000055555555194 in मुख्य () at ./tempfil0123.tmphin.c:15
15      म_लिखो("1 से 10 तक का योग = %d\n", जोड़ो(10));
Value returned is $1 = 55
$2 = 55

$ addr2line -f -s -e yog-g 0x0000000000001149
जोड़ो
tempfil0123.tmphin.c:6
```

Three layers do three jobs here. The front end lowers keywords to C, where they vanish into control flow. The symbol table and the DWARF information carry every name in Romenagri, byte for byte, through the assembler and the linker into the ELF file, and from there to `nm`, `addr2line`, GDB, OpenOCD on a JTAG or SWD probe, or a kernel's panic trace. A presentation layer applies the Romenagri inverse wherever a person reads: a GDB front end, an IDE panel, a filter on `addr2line` or on a panic trace. No link in the chain has to accept anything but letters, digits and the underscore, and nothing is lost, because the map is bijective. Drop the script layer and pass names as raw UTF-8, or flatten them into names that cannot be mapped back, and some link in that chain either rejects them or hands the developer names that no longer match the source.

Two details are worth knowing. GDB lists the generated C because the guru shaili does not emit `#line` directives; with them, the same session would list the Hindi source line by line. And Romenagri writes an independent vowel with a leading underscore, so a vowel-initial name at file scope, `_aapakaa` for `आपका`, begins with an underscore, which the C standard reserves for the implementation at that scope; GCC accepts it, and a project that exports such symbols can fix a prefix for them.

The Urdu edition, run

`zistgah/urdu-ilm` carries the same shailis for Urdu. The guru shaili's lexer is written in Urdu, `h2c_urdu.uhin`, and the driver `urducc` sends an Urdu program through `fixuninum` and `fixarabnum` for the digits, `fltr_ur_hi` to the Devanagari hub, `uni2acii` and `acii2cf` to Romenagri, then `h2c_urdu` and the standard Hindawi driver. The run below compiles the Urdu edition's own C sample:

File: `vendor/urdu-ilm/ILM/samples/UrduC.uhin`

```
<شیلی گرو>
#شامل <معیاری.س>
صحیح مرکزی
{
    حرف ن[80];
    صحیح ک
    لکھو ("آپ کا نام کیا ہے؟\n");
    لکھو ("س", ن);
    لکھو ("س.\n", ن);
    برائے (ک=1; ک>=10; ++ک)
        لکھو ("%d\n", ک);
    واپس 0
}
```

Executed while this guide was built: `bash examples/hindawi.sh urdu`

```
$ urducc UrduC.uhin
संकलन के परिणाम
=====

The C that reached gcc:
#include <stdio.h>
int main()
{
    char na[80];
    int ka;
    printf("आप का नाम क्या है?\n");
    scanf("%s", na);
```

```

printf("इलसलाम एलीकम %s.\n",na);
for(ka=1; ka<=10; ka++)
    printf("%d\n",ka);
return 0;
}

$ ./hin.exe, with Ali typed at the prompt
आप का नाम क्या है?
इलसलाम एलीकम Ali.
1
2
3
4
5
6
7
8
9
10

```

The keywords are C and the names are Romenagri, reached through the hub: ن is na and ک is ka. The strings print in the hub script and show the frontier of Chapter 19 at work: کیا, written without its short vowel, arrives as क्या rather than क्या, and ہے as हे rather than है. Rendering them back into Urdu is the reverse filter's job.

ILM's construct model

In the merged release, `ilm/constructs.csv` holds 39 constructs in 27 human languages and `ilm/decorators.csv` holds 216 realisations of them in 8 host languages; a keyword table of 1,801 constructs in 27 languages generates the Hindawi distributions. The construct is primary: a counted loop is translated once per human language, not once per host language, and a decorator records what a host requires, such as `stdio.h` for printing in C. The construct model is the keyword layer, the layer `h2c` implements for C. Names still pass through the script layer, Romenagri, exactly as above.

22. Below the software: logic, chips, ternary and qubits

In plain words. What happens inside chips: switches, circuits, three-valued logic, and the very cold machines that hold qubits.

From logic to CMOS

Every digital function reduces to Boolean algebra, and NAND alone is enough to build all of it. In CMOS, a network of p-type transistors pulls the output up and a dual network of n-type transistors pulls it down, so that in a settled state no current flows from supply to ground. Power is spent mainly in switching, $P \approx \alpha C V^2 f$, with α the fraction of nodes switching per cycle, C the capacitance switched, V the supply voltage and f the clock frequency; the V^2 is why every generation of chips has lowered its voltage.

How a chip is designed

1. Describe the hardware at register-transfer level in Verilog, VHDL or SystemVerilog, or generate it from Chisel or Amaranth.
2. Simulate it, with Icarus Verilog or Verilator.
3. Synthesise it into a netlist of standard cells, with Yosys.
4. Floorplan, place, build the clock tree and route, with OpenROAD.
5. Check design rules and compare layout against schematic, with Magic, KLayout and Netgen.
6. Write the layout as GDSII or OASIS and send it to the foundry: the tapeout.
7. Fabricate, test and package.

Open process design kits let anyone take this path to real silicon: SkyWater SKY130 (130 nm), GlobalFoundries GF180MCU (180 nm) and IHP SG13G2 (130 nm, SiGe BiCMOS). [zistgah/jugaad28](https://www.zistgah.com/jugaad28) catalogues integrated circuits taped out on 28 nm, with evidence-gated descriptors and a low-cost open laboratory. A hardware description language is a language like any other: its front end can be given native-language keywords by the three-axis method of Chapter 21, and what reaches the synthesiser is a netlist, not words.

Instruction sets and the boot chain

The instruction set is the contract between software and hardware. RISC-V is an open one: a small base, RV32I or RV64I, with standard extensions for multiplication (M), atomics (A), floating point (F and D), compressed instructions (C) and vectors (V), which anyone may implement without a licence fee. An open instruction set, open design tools and reproducible toolchains carry the argument of Chapter 21 down to the silicon.

A computer starts through a chain of programs: a boot ROM, the firmware (BIOS or UEFI), a boot loader such as GRUB, the kernel, the first user process such as systemd, and then everything else. Each is a program in some language, and each can be given a native-language front end without changing what the hardware executes; the hindawiai organisation keeps a fork of the Linux kernel tree, [hinlin](https://github.com/hindawiai/linux), for exactly this work.

Balanced ternary: Zamin and PRATIK

Balanced ternary uses the digits -1 , 0 and $+1$, written T, 0 and 1:

$$n = \sum_i t_i 3^i, \quad t_i \in \{-1, 0, +1\}.$$

It needs no sign: a number is negated by flipping every digit, and rounding is simply truncation. It is also the most economical integer base. Representing numbers up to N in base b takes about $\log_b N$ digits of b states each, a cost proportional to

$$b \log_b N = \frac{b}{\ln b} \ln N,$$

which is least at $b = e \approx 2.718$; among whole numbers, 3 wins over 2 and 4. The Setun computer, built at Moscow State University in 1958, ran on balanced ternary.

[zistgah/zamin](#) is the physical realisation of the PRATIK balanced-ternary substrate for the PEDLER event algebra. Its poised zero is a physical high-impedance ground, so that the electrical ground reference and the computational zero are the same node, and its spawning operator creates new physical dimensions on a memristive three-dimensional crossbar. The repository holds SPICE cells, an interactive circuit simulator and a paper; [zistgah/pratik_core_mvp](#) is the kernel in C++ and CUDA.

Executed while this guide was built: `python3 examples/ternary.py`

```
5 =      1TT   negated by flipping every digit:      T11 = -5
-5 =      T11   negated by flipping every digit:      1TT = 5
13 =      111   negated by flipping every digit:      TTT = -13
40 =      1111  negated by flipping every digit:      TTTT = -40
-121 =     TTTT  negated by flipping every digit:     11111 = 121
2026 =    10T10001 negated by flipping every digit:    T01T000T = -2026
```

```
radix economy b/ln b for b = 2.000: 2.8854
radix economy b/ln b for b = 2.718: 2.7183
radix economy b/ln b for b = 3.000: 2.7307
radix economy b/ln b for b = 4.000: 2.8854
radix economy b/ln b for b = 10.000: 4.3429
```

Superconducting qubits

A Josephson junction is two superconductors separated by a thin insulator, usually aluminium, aluminium oxide and aluminium. Its current and voltage obey

$$I = I_c \sin \varphi, \quad V = \frac{\hbar}{2e} \frac{d\varphi}{dt},$$

where φ is the phase difference across it. It behaves as a nonlinear inductor, and the nonlinearity is what makes the lowest two energy levels of a circuit usable as a qubit. The transmon, the most common design, has the Hamiltonian

$$H = 4E_C (\hat{n} - n_g)^2 - E_J \cos \hat{\varphi},$$

and is operated with E_J/E_C around 50, which makes it nearly insensitive to stray charge (Koch and colleagues, 2007). Its transition frequency lies around 4 to 8 GHz.

The cold follows from that frequency. Thermal energy has to be far below the energy of one microwave photon at the qubit frequency. At 10 mK, which is -273.14°C , $k_B T/h$ is about 0.21 GHz, well below 5 GHz; at the 4 K of liquid helium it would be about 83 GHz, and the qubit would be swamped by thermal excitation. Hence the dilution refrigerator, the large gold-coloured structure in photographs of quantum computers; the processor itself is a small chip at its bottom.

That chip is an integrated circuit: aluminium or niobium films on silicon or sapphire, patterned by lithography from a layout file like any other chip, through processes related to, but distinct from, CMOS logic processes. Qiskit Metal is IBM's open tool for laying out and analysing such chips. Other ways to build qubits are trapped ions, neutral atoms, photons and spins in silicon; nitrogen-vacancy centres in diamond keep their spin coherence at room temperature and are used today as sensors.

PANINIq's oscillator substrate, Chapter 23, is different in kind: a classical network of coupled oscillators that can be built from ordinary electronics and behaves as an Ising machine. Its hardware bridge is mocked today; its mathematics is not.

23. Quantum computing and PANINIq, with the mathematics

In plain words. How quantum computers work, and how a network of swinging oscillators can solve the same kind of puzzle.

States, gates and measurement

A qubit's state is a unit vector in $\mathbb{C}^2$,

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1,$$

which, up to a global phase, is a point on the Bloch sphere: $|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\phi} \sin \frac{\theta}{2} |1\rangle$. The state of n qubits lives in the tensor product of n copies, a space of 2^n complex amplitudes. Gates are unitary matrices:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad \text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

A Hadamard followed by a CNOT entangles two qubits: $\text{CNOT}(H \otimes I)|00\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Measuring in the computational basis gives outcome x with probability $|\langle x|\psi\rangle|^2$, the Born rule. Between measurements the state evolves under a Hamiltonian H by $i\hbar \frac{d}{dt}|\psi\rangle = H|\psi\rangle$, so that, in units with $\hbar = 1$,

$$|\psi(t)\rangle = e^{-iHt} |\psi(0)\rangle.$$

Simulating n qubits exactly means storing 2^n amplitudes of 16 bytes each: 30 qubits need 16 GiB. That is why a laptop simulates up to about thirty qubits, and why the encoding of a problem into qubits matters so much.

Ising, QUBO and Max-Cut

An Ising problem assigns spins $s_i \in \{-1, +1\}$ to minimise

$$E(s) = - \sum_{i < j} J_{ij} s_i s_j - \sum_i h_i s_i.$$

With $x_i = (1 - s_i)/2 \in \{0, 1\}$ the same problem becomes a quadratic unconstrained binary optimisation (QUBO). Max-Cut asks for the split of a weighted graph's vertices into two sides that maximises the weight of the edges crossing between them:

$$C(s) = \sum_{(i,j) \in E} w_{ij} \frac{1 - s_i s_j}{2}.$$

Each term is w_{ij} when the edge is cut and 0 when it is not, so maximising C is minimising $\sum w_{ij}s_i s_j$, the energy of an anti-ferromagnet with $J_{ij} = -w_{ij}$. Max-Cut is NP-hard; the best classical guarantee in polynomial time, Goemans and Williamson's semidefinite relaxation with random rounding, reaches 0.878 of the optimum.

QAOA

The Quantum Approximate Optimisation Algorithm prepares

$$|\gamma, \beta\rangle = \prod_{k=1}^p e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |+\rangle^{\otimes n}, \quad H_C = \sum_{(i,j) \in E} w_{ij} \frac{1 - Z_i Z_j}{2}, \quad H_M = \sum_i X_i,$$

and a classical optimiser adjusts the $2p$ angles to maximise $\langle \gamma, \beta | H_C | \gamma, \beta \rangle$; measuring the final state gives candidate cuts. With $p = 1$ on 3-regular graphs, the expected cut is at least 0.6924 of the optimum (Farhi, Goldstone and Gutmann, 2014). QAOA uses one qubit per vertex, so exact simulation costs 2^N amplitudes; PANINIq's QAOA baseline therefore stops at $N \leq 12$ and uses the COBYLA optimiser.

The continuous-time quantum walk, compactly encoded

A continuous-time quantum walk (Farhi and Gutmann, 1998) evolves a state over the vertices of a graph under a Hamiltonian built from the graph: $|\psi(t)\rangle = e^{-iHt}|\psi(0)\rangle$. PANINIq encodes the N vertices as the basis states of $n = \lceil \log_2 N \rceil$ qubits, a binary address, so its Hilbert space has $2^n < 2N$ dimensions rather than 2^N . Its Hamiltonian is the coupling matrix placed in the top-left block of a $2^n \times 2^n$ zero matrix, and the evolution is one arbitrary unitary gate:

```
# pedler/quantum_validator.py in zistgah/paniniq, commit 82a5cf2 (excerpt)
num_qubits = max(1, int(np.ceil(np.log2(max(N, 2)))))
dim = 2 ** num_qubits
H = np.zeros((dim, dim), dtype=complex)
H[:N, :N] = K_matrix
...
U_t = expm(-1j * H * evolution_time)
qc.unitary(Operator(U_t), list(range(num_qubits)), label="CTQW(K,t)")
```

The test suite checks that the circuit's statevector equals e^{-iHt} applied directly. A general unitary on n qubits compiles into $O(4^n)$ elementary gates, so this is a construction for validating the classical dynamics, not an efficient circuit for hardware; the repository says the same.

The Jelly Substrate: coupled phase oscillators

The substrate is a network of phase oscillators obeying the Kuramoto equations, with coupling that decays with distance:

$$\frac{d\theta_i}{dt} = \omega_i + \sum_j K_{ij} \sin(\theta_j - \theta_i), \quad K_{ij} = K_0 e^{-\alpha r_{ij}}, \quad K_{ii} = 0.$$

```
# pedler/substrate.py in zistgah/paniniq, commit 82a5cf2 (excerpt)
def kuramoto_rhs(self, t, theta):
    phase_diffs = theta[None, :] - theta[:, None]
    return self.omegas + np.sum(self.K * np.sin(phase_diffs), axis=1)
```

How synchronised the network is shows in the order parameter,

$$R e^{i\psi} = \frac{1}{N} \sum_{j=1}^N e^{i\theta_j}, \quad 0 \leq R \leq 1,$$

near 1 when the phases lock together and near 0 when they are scattered. For identical all-to-all coupling K/N , each oscillator feels only the mean field, $\dot{\theta}_i = \omega_i + KR \sin(\psi - \theta_i)$, and synchrony appears once K exceeds $K_c = 2/(\pi g(0))$, where g is the symmetric, single-peaked distribution of natural frequencies; for a Lorentzian of half-width γ , $K_c = 2\gamma$.

Oscillators as an Ising machine

Coupled oscillators can minimise an Ising energy directly (Wang and Roychowdhury, 2019). With an added second-harmonic injection of strength K_s ,

$$\frac{d\theta_i}{dt} = -K \sum_j J_{ij} \sin(\theta_i - \theta_j) - K_s \sin(2\theta_i),$$

the function

$$\mathcal{E}(\theta) = -K \sum_{i,j} J_{ij} \cos(\theta_i - \theta_j) - K_s \sum_i \cos(2\theta_i)$$

never increases along the motion, so the network settles into its minima. The injection pushes each phase to 0 or π , where $\cos(\theta_i - \theta_j) = s_i s_j$ with $s_i = \cos \theta_i = \pm 1$, and $\mathcal{E}$ becomes the Ising energy. PANINIq's Max-Cut solver uses the simplest form of this: it sets the coupling to minus the adjacency, so that neighbours repel in phase, relaxes, and reads each spin as the sign of $\cos \theta_i$. Its tests check the result against brute force on six graphs: never above the optimum, and exact on the bipartite ones.

```
# pedler/maxcut.py in zistgah/paniniq, commit 82a5cf2 (excerpt)
sub.K = -1.0 * adjacency          # connected nodes repel in phase
...
spins = np.sign(np.cos(final_phases))
```

The PEDLER automaton on the substrate

PANINIq implements PEDLER as a six-tuple $P = (I, G, U, S, F, *)$. U is the stream of point events, vectors in $\mathbb{R}^3$. S is the set of states, one per oscillator, and F maps each state to a point in $\mathbb{R}^3$. G is the resolution-match kernel, $\exp(-\gamma \|F(v_i) - r_t\|)$, the similarity between an event r_t and each state. I is the inclination field over the states, a historical prior plus the weighted resolution match, computed afresh for each event. The operator $*$ governs growth: when an event is presented and the relaxed network fails to lock, its order parameter falling below the novelty threshold, a new state is spawned;

on a separate schedule, faded states are pruned and strongly locked ones consolidated. “Inclination field”, “resolution match” and “point event” are the framework’s own terms; the code implements the equations as given and claims nothing more about cognition. The repository also records one open question plainly: PEDLER’s canonical inclination is signed, lives on edges and lies in $[-1, +1]$, while this code’s is non-negative and lives on nodes, and which one the substrate should carry is the author’s decision.

What is tested, and what is open

From the repository’s own status table: the Jelly Substrate, the PEDLER engine, the Max-Cut solvers, the frame packing of the hardware bridge, the Hamiltonian and unitary mathematics, the Qiskit circuit and QAOA are **tested**; the PyTorch substrate is **written** and not executed; the hardware port is **mocked**; a Duffing extension with amplitude as well as phase, a real firmware protocol, and the larger system running from brain signals through symbolic orchestration to a physical instruction set and manufacturing are **planned**. The run in Chapter 14 shows all of it: states spawning as the order parameter drops, a three-qubit walk whose statevector sums to one, and the oscillator, brute force and QAOA agreeing on a cut of 4.

Exercises

1. Compute $H|0\rangle$, $H|1\rangle$ and HZH .
2. Show that $\frac{1}{2}(1 - \mathbf{s}_i \mathbf{s}_j)$ is 1 exactly when an edge is cut.
3. For a graph with 12 vertices, how many qubits do the walk and QAOA each need, and how many amplitudes does each simulation store?
4. Differentiate $\mathcal{E}$ along the motion and show that $d\mathcal{E}/dt \leq 0$ when $\mathbf{J}$ is symmetric.
5. In PANINIq, change the novelty threshold and predict, before running, how the number of spawned states changes.

Appendices

D. For researchers in science, technology, engineering and mathematics

In plain words. A guide for researchers in science, technology, engineering and mathematics: what can be read, run and tested now, and what is still open.

Each field below lists what in the estate can be read, run and tested now, and what is open. Status words keep their meaning from Chapter 1: tested, executed, written, mocked, planned.

Computer science

Programming languages and compilers. Hindawi's composed transducers over unmodified host toolchains; the Romenagri symbol bridge through ELF, DWARF and GDB; the PANINI construct model, with 39 constructs in 27 languages and 216 decorators across 8 host languages; the bootstrap and its self-hosting fixed point (Chapters 21 and 6). Executed here. **Open:** `std2hin` doubles every underscore, so vowel-initial names do not round-trip back from C; the guru shaili emits no `#line` directives; the YACC shaili awaits a fresh conformance run.

Systems. The hinlin kernel tree and the mass conversion of real C sources, including kernel trace code, into Hindi and back (`hindawiai/chintamani`, `Romenagri/mass_hindawi`).

AI and machine learning. The cyclers and GENIE as AI-agnostic, human-inspected method (Chapter 8); Romenagri as a tokenisation substrate, where bytes fall 1.76 times and the alphabet from 51 to 24 symbols without loss, but token counts did not fall at matched merges, so the predicted gain is still to be shown; COPA's Authority Projection Index as a measurable property of a model's behaviour.

Software engineering. Executable contracts and gates in every repository; the provenance chain, Tok DOI, spiguard, Candor and Misty DOI; VGC; the component cards of Chapter 15, each the result of a real run.

Data. ILM's registry of 7,867 languages and 226 scripts; 13,668 keyword standards; the corpora in the Romenagri tree.

Electrical and electronic engineering

The open chip-design flow and the 28 nm tape-out catalogue, jugaad28, with its evidence-gated descriptors (Chapter 22); Zamin's ternary cells in SPICE, whose testbench needs attention on ngspice 42 (Chapter 15); the PRATIK kernel, whose CPU build passes and whose CUDA build needs an NVIDIA toolchain; device control on ESP32, zasab, built but not yet pushed.

Physics

QEDLER, the event-hypergraph research programme for physics, self-declared as a framework with open problems; oscillator Ising machines and Kuramoto dynamics in PANINIq, with their Lyapunov function (Chapter 23); superconducting qubits and the physics of the cold (Chapter 22).

Mathematics

Coding theory for reversible transliteration: Kraft and McMillan, and Sardinas and Patterson (Chapter 19); formal languages and rewriting in Pāṇini's grammar, with pratyāhāras as interval classes over an ordered list (Chapter 20); graph theory in the typed hypergraph and in Max-Cut; dynamical systems in the order parameter and in the oscillators' energy; the formal tuples of the CEM kernel, $\mathcal{E} = (E, \mathcal{C}, \Pi, W)$, and of the AyeAI Triad.

Biology and medicine

PANINIB, the biological realization arm, which lowers intent to sequence IR and claims no wet-laboratory result; CEMb, operated through Interglial Healthcare; the Quantum Neuromorphic BCI programme, with brain-to-text, cognitive prosthetics and intelligent rehabilitation as directions needing clinical collaboration; VIKRAM's Health section and its tools for an accessible virtual hospital; the earlier HMSEI wearable and the Dr Rho telepresence platform.

Cognitive science and neuroscience

PEDLER's two branches, through LVF and MLCNE to the cognitive enablement modules, and through NI2A2 to ANGEL; its primitives; the brain-computer interface work of Act III.

Astronomy and the earth and space sciences

CHAKRA, the offline observatory and calendars, whose JavaScript and C99 versions produce byte-identical output; Jyotish, derived from it; the habitats, with a lander embodiment on the Moon and a quadrotor on Mars inside the shared dome.

Robotics and manufacturing

PANINIphy, the physical realization arm, which compiles physical intent and chooses parts by force, stroke and voltage; pench, the embodied cycler; † the modular self-reconfiguring robots of the notes.

© 1993–2026 Abhishek Choudhary. All rights reserved. AyeAI. Text CC BY-SA 4.0; scripts GPL-3.0-or-later. Version 2.0.0, 3 October 2026.